

Inventory-Deus

Authentication and Authorization — the runtime companion to the table-connections document.

Scope of this document

The connections document explains the *data shape* of the auth schema and how the `auth.app_user.id` primary key is referenced by operational tables across the system. This document explains the *runtime side*: how a human becomes a logged-in user, how that login becomes a token, how the token is validated by every service on every request, and how the user identifier finally lands in the `user_id` column of an audit row. Together the two documents describe the complete security story — model and motion.

Part 1 — The actors

Five distinct components participate in every authenticated request. Knowing what each one does and does not do is the foundation of the rest of the document.

Actor	Responsibility	What it does NOT do
Browser / Client	Holds the JWT in <code>sessionStorage</code> , attaches it to every API call as <code>Authorization: Bearer <token></code> , redirects to <code>/login.html</code> on 401.	It does not validate the token itself, and it does not store the password after login.
API Gateway (:9000)	The single front door. Serves <code>/login.html</code> , <code>/menu.html</code> , the per-service dashboards via <code>/ui/{svc}/**</code> , and routes API calls. Optionally validates JWT signature before forwarding.	It does not issue tokens and it does not check fine-grained permissions.
Auth-service (:9001)	The only service allowed to write to the auth schema. Owns the login endpoint, password hashing (<code>bcrypt</code>), JWT issuance, <code>user/role/permission</code> CRUD.	It does not enforce permissions on other services' endpoints — it only tells them who the user is and what they may do.
Downstream services (9002-9016)	On every request, parse the JWT, verify the signature with the shared secret, extract claims, and decide whether this user may perform this action against this resource.	They do not call back to auth-service on every request. The JWT is self-contained.
Eureka (:9017)	Service registry only. Lists which services are alive.	It plays no role in authentication. A service registered with Eureka is not automatically trusted.

Part 2 — The JWT itself

A JSON Web Token is three Base64-encoded parts joined by dots: `header.payload.signature`. Anyone can decode the first two parts; only someone holding the shared secret can produce a valid signature for the third part. That is the entire trust model in one sentence.

2.1 What the token looks like in transit

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI3IiwiaWZw1haWwiOiJzYW1AYWNTZS5jb20iLCJyb2xlcYI6WyJCVVlFUiJdLCJ3aHMiOiLsxDNnLCJpYXQiOiJlE3NDcwMDAwMDAsImV4cCI6MTc0NzAwMzYwMH0.k9Q...signature_bytes...
```

2.2 The decoded header

```
{
  "alg": "HS256",          // HMAC-SHA256, symmetric – both signer and verifier share one secret
```

```
} "typ": "JWT"
```

2.3 The decoded payload (the claims)

```
{
  "sub": "7", // subject – the auth.app_user.id
  "email": "sam@acme.com", // human-readable identity
  "roles": ["BUYER", "WAREHOUSE_OPS"], // names of auth.role rows
  "perms": ["po.create", "po.submit", "stock.read"], // optional, computed at login
  "whs": [1, 3], // warehouse_ids from auth.warehouse_access
  "iat": 1747000000, // issued at (UNIX seconds)
  "exp": 1747003600 // expires at – 1 hour later
}
```

Every other service trusts these claims as long as the signature verifies. That is why the secret must never leak — anyone with it can forge a token and become any user.

2.4 The shared HMAC secret

A single random string (at least 256 bits) configured identically in every service through an environment variable or a profile property. Suggested key name:

```
# .env (NEVER committed)
INVENTORY_JWT_SECRET=z9p2W...VeryLongRandomString...e3kQ

# application.properties in each service
security.jwt.secret=${INVENTORY_JWT_SECRET}
security.jwt.expiry-minutes=60
```

Hard rule: the JWT secret never appears in source control, never appears in logs, never appears in client-side code. If it leaks, rotate it (change the env var and restart everything) — all existing tokens become instantly invalid, which is exactly what you want.

Part 3 — The login flow, step by step

A complete trace from the moment Sam types her password to the moment her token is sitting in sessionStorage. Each step lists the actor, the data flowing, and what could go wrong.



3.1 What the auth-service does internally

- | | |
|-------------------------------|--|
| 1. Validate input shape | (email present, password present) |
| 2. Lookup user | SELECT id, email, password_hash, active
FROM auth.app_user WHERE email = ? |
| 3. Reject if inactive | throw 401 |
| 4. Verify password | BCrypt.matches(rawPwd, row.password_hash) |
| 5. Load roles | SELECT r.name FROM auth.role r
JOIN auth.user_role ur ON ur.role_id = r.id
WHERE ur.user_id = ? |
| 6. Load permissions | SELECT p.code FROM auth.permission p
JOIN auth.role_permission rp ON rp.permission_id = p.id
WHERE rp.role_id IN (...) |
| 7. Load accessible warehouses | SELECT warehouse_id FROM auth.warehouse_access
WHERE user_id = ? |
| 8. Build JWT claims, sign | Jwts.builder(...).signWith(secret, HS256).compact() |
| 9. Return 200 { token, user } | |

Steps 5-7 happen at login because the claims live inside the token. Loading them once at login means downstream services never have to ask auth-service "what may this user do?" — the answer is already in the JWT they sent.

Part 4 — How the gateway uses the token

The API gateway has two jobs around auth: serving the static portal pages, and validating tokens on inbound API calls. It explicitly does NOT do fine-grained permission checks — those live in the owning service.

4.1 Two route categories

Route pattern	Auth check at gateway?
/login.html, /menu.html, /ui/**, /js/**, /css/**	No. Static resources are open; the page itself enforces login client-side by reading sessionStorage.
/auth/login, /auth/register	No. These are how you become authenticated — gating them would be a chicken-and-egg.
/health/**	No. Service health pings are public so the menu tiles can light up before login.
/auth/me, /auth/users/**	Yes — JWT signature verified, claims attached to forwarded headers.
/purchase-orders/**, /sales-orders/**, /inventory/**, ... (all domain APIs)	Yes — JWT signature verified. If invalid or expired, return 401 without forwarding.

4.2 The gateway filter chain (conceptual)

```
Incoming request
|
v
[1] Is the path in the public allow-list? -- yes --> forward as-is
| no
v
[2] Pull "Authorization: Bearer <token>" header
| missing -> 401
v
[3] Verify HMAC signature using INVENTORY_JWT_SECRET
| invalid -> 401
v
[4] Check exp claim
| expired -> 401
v
[5] Inject claims into forwarded request as headers:
    X-User-Id: 7
    X-User-Email: sam@acme.com
    X-User-Roles: BUYER, WAREHOUSE_OPS
    X-User-Warehouses: 1,3
v
Forward to downstream service
```

Injecting parsed claims as headers (in addition to leaving the raw Bearer token in place) is a convenience — downstream services may parse the JWT themselves, but they can also just trust the gateway-issued headers as long as the network between them is private. For class projects, parsing in each service is the cleaner pattern.

Part 5 — Per-service authorization

The gateway proves *who* the user is. Each downstream service decides *whether* they may do the requested action. This split is what keeps the gateway thin and the rules close to the data they protect.

5.1 Three levels of check, in order

Level	Question	How enforced
1. Authentication	Is this request signed by our secret?	JwtAuthFilter in SecurityConfig. Rejects with 401 if signature invalid, missing, or expired.
2. Authorization (role/permission)	Does this user hold the permission required by this endpoint?	Spring Security @PreAuthorize annotation on the controller method, reading from claims.
3. Resource scoping (warehouse access)	May this user touch <i>this specific</i> warehouse / order / supplier?	Method-level guard that compares the resource's warehouse_id against claims.whs. Throws 403 if mismatch.

5.2 A worked example — approving a purchase order

Sam (user 7, role BUYER, warehouses [1, 3]) hits POST /purchase-orders/44/approve. The procurement-service controller runs these checks in order:

```
@PostMapping("/purchase-orders/{id}/approve")
@PreAuthorize("hasAuthority('po.approve')") // <-- Level 2: permission check
public ResponseEntity<ApiResponse<Void>> approve(
    @PathVariable Long id,
    @AuthenticationPrincipal JwtPrincipal user // <-- claims injected here
) {
    PurchaseOrder po = poRepo.findById(id)
        .orElseThrow(() -> new NotFoundException("PO " + id));

    // Level 3: this user may only approve POs for warehouses they own
    if (!user.warehouseIds().contains(po.getWarehouseId())) {
        throw new ForbiddenException("not your warehouse");
    }

    poService.approve(po, user.userId()); // writes user_id into audit columns
    return ResponseEntity.ok(ApiResponse.success(null));
}
```

5.3 What goes wrong, and what status is returned

Situation	Where it is caught	HTTP status
No Authorization header	Gateway	401 Unauthorized
JWT signature does not verify	Gateway	401 Unauthorized
JWT exp claim is in the past	Gateway	401 Unauthorized
Token valid but user lacks po.approve	@PreAuthorize in service	403 Forbidden
User has permission but wrong warehouse	Controller method body	403 Forbidden
PO id does not exist	Service after the auth checks pass	404 Not Found

The 401 vs 403 distinction matters. 401 means "I do not know who you are" — the client should clear sessionStorage and bounce to /login.html. 403 means "I know who you are and you are not allowed" — the client should show an error toast, not a re-login.

Part 6 — The audit trail (closing the loop with the data model)

Every operational write carries the user's id in a `user_id`, `created_by`, `approver_id` or similar column. This is what ties the runtime auth context to the foreign-key map in the connections document.

6.1 Where `user_id` lands

Table	Column	Captured during
inventory.stock_ledger	user_id	every quantity change
procurement.purchase_order	created_by	PO creation
procurement.approval	approver_id	PO approval
sales.sales_order	created_by	SO creation
movement.movement	user_id	transfer / adjustment
receiving.qa_check	inspector_id	QA inspection
fulfillment.pick_list	picker_id	pick assignment
audit.count_session	counter_id	count opening
audit.count_approval	approver_id	variance approval
notification.subscriber	user_id	subscription setup
reporting.export_job	user_id	export request

6.2 How the `user_id` gets there

The flow from JWT claim to database column, in five lines of Java:

```
// In SecurityFilterChain – parse JWT once, store principal in SecurityContext
JwtPrincipal principal = jwtParser.parse(bearerToken);
SecurityContextHolder.getContext().setAuthentication(
    new UsernamePasswordAuthenticationToken(principal, null, principal.authorities()));

// In the service method – read it back when writing the row
po.setCreatedBy(principal.userId());      // <-- this is the FK to auth.app_user(id)
poRepo.save(po);
```

Because the column is a real foreign key, you cannot accidentally write `created_by = 99999` for a user that does not exist. The database refuses. The auth model and the data model police each other.

6.3 A query that uses both ends of the system

"Who has approved purchase orders over **1,000,000** in the last 30 days?"

```
SELECT  u.email,
        COUNT(*)      AS approvals,
        SUM(po.total) AS total_value
FROM    procurement.approval ap
JOIN    procurement.purchase_order po ON po.id = ap.po_id
JOIN    auth.app_user      u      ON u.id = ap.approver_id
WHERE   po.total > 1000000
        AND ap.created_at >= now() - INTERVAL '30 days'
GROUP BY u.email
ORDER BY total_value DESC;
```

Three schemas joined by primary-key/foreign-key links. The auth identity that was created at login (Part 3) is the same id stored in `approver_id` (Part 6) and the same id you query here. One id, end-to-end.

Part 7 — Logout, expiry, and refresh

7.1 Logout

JWTs are stateless — there is no server-side session to destroy. "Logout" is a client-side action: clear `sessionStorage`, redirect to `/login.html`. The token, if anyone still has it, will continue to work until exp.

```
function logout() {
  sessionStorage.removeItem("inv_jwt");
  sessionStorage.removeItem("inv_user");
  window.location.href = "/login.html";
}
```

For a class project this is acceptable. For a real product you would add a server-side denylist of revoked tokens, checked by the gateway on every request. Out of scope here.

7.2 Expiry

Tokens expire one hour after issue (configurable via `security.jwt.expiry-minutes`). When a service receives an expired token it returns 401. The client's `apiFetch` wrapper sees the 401 and bounces to `/login.html` automatically.

```
async function apiFetch(url, opts = {}) {
  const token = sessionStorage.getItem("inv_jwt");
  const res = await fetch(url, {
    ...opts,
    headers: { ...opts.headers, "Authorization": "Bearer " + token }
  });
  if (res.status === 401) {
    sessionStorage.clear();
    window.location.href = "/login.html";
    return;
  }
  return res;
}
```

7.3 Refresh (the grown-up option)

A second long-lived "refresh token" can be issued alongside the access token; the client trades it for a fresh JWT a minute before exp. For a class project, 1-hour tokens with re-login on expiry is fine. Documented here so you know the next step if the grader asks.

7.4 Tab close = logout

Because the token lives in `sessionStorage` (not `localStorage`), closing the browser tab destroys it. The next tab opens fresh and asks for login. This was a deliberate v1 decision and we keep it.

Part 8 — Hard rules

Passwords are never stored — only their bcrypt hash.

A leaked database should reveal no password. Bcrypt is one-way and salted.

The JWT secret is at least 256 bits of random and is identical across all services.

A different secret per service breaks verification. A short secret is brute-forceable.

The secret lives in an environment variable, never in source control.

Add `.env` to `.gitignore`. Use `.env.example` with a fake value to document the variable.

Permissions live in claims, not in DB lookups on every request.

A request handler that calls auth-service to ask "may they do X?" defeats the point of JWTs and adds a hard dependency. Permissions are computed once at login and travel with the token.

Warehouse scoping is checked in every domain service, not the gateway.

Whether user 7 may touch warehouse 5 depends on the resource being touched. Only the owning service knows the resource's warehouse_id.

Every operational write records the user_id from the JWT subject.

No anonymous writes. If a row exists you can prove who created it.

401 and 403 mean different things — never collapse them.

401 -> client re-authenticates. 403 -> client shows an error. Mixing them either kicks users out unnecessarily or silently hides authorization failures.

The /auth/login endpoint is rate-limited.

Even at class-project scale, the principle stands: brute-force protection lives at the one place credentials are accepted.

Tokens are short-lived.

One hour is generous. Anything longer than a workday is asking for trouble.

Eureka registration is not authentication.

A rogue service registering with Eureka must not be trusted just because it appears in the registry. Trust comes from JWT verification, not service discovery.

Part 9 — End-to-end walkthrough

Putting every part of the document together with one realistic story. Sam, a buyer with access to warehouses 1 and 3, approves PO-44 (worth ■2,500,000, ship-to warehouse 3).

1. Browser POST /auth/login { "sam@acme.com", "..." }
2. Gateway forwards to auth-service
3. Auth-svc bcrypt-verifies password, loads roles + perms + whs from DB
4. Auth-svc signs JWT { sub:7, roles:[BUYER], perms:[po.approve], whs:[1,3], exp:+1h }
5. Gateway returns 200 { token, user } to browser
6. Browser stores token in sessionStorage, redirects to /menu.html
7. Browser opens procurement-dashboard via /ui/procurement/...
8. Browser POST /purchase-orders/44/approve (Bearer eyJ...)
9. Gateway verifies signature, extracts claims, injects X-User-* headers, forwards
10. proc-svc JwtAuthFilter parses token, builds JwtPrincipal
11. proc-svc @PreAuthorize("hasAuthority('po.approve')") passes
12. proc-svc controller checks po.warehouse_id (3) is in user.whs ([1,3]) -> OK
13. proc-svc inserts procurement.approval(po_id=44, approver_id=7, created_at=now)
14. DB FK approver_id REFERENCES auth.app_user(id) verified -- accepted
15. proc-svc updates purchase_order.status = APPROVED
16. proc-svc writes shared.event_outbox(event_type="po.approved", ...)
17. proc-svc returns 200

Hours later, an auditor runs:

```
SELECT u.email, ap.created_at FROM procurement.approval ap
JOIN auth.app_user u ON u.id = ap.approver_id WHERE ap.po_id = 44;
-> "sam@acme.com, 2026-05-12 14:03"
```

The chain is complete: login -> JWT -> verified -> permission -> scope -> FK-backed audit row.

Every link in that chain is enforced by something concrete: bcrypt at login, HMAC at the gateway, Spring Security at the controller, the application code at the resource check, and Postgres at the foreign-key constraint. No link is "trust the client" or "remember to check this". That is what makes the design defensible.